TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS • CAPÍTULO 5

Python e PostgreSQL

Conectar um programa ao banco na nuvem, com segurança, e construir um CRUD de verdade

```
banco = Banco()
linhas = banco.consultar(
    "SELECT nome_produto FROM produtos WHERE preco < %s",
    (100,),
)
```

PROFESSOR
Tarik Ponciano

Roteiro

01 **Python conversa com o banco**
O arquivo `.env`, o psycopg e o primeiro script: apagar, criar e preencher o banco

02 **Uma classe para o banco**
A conexão num lugar só e as consultas com parâmetros

03 **CRUD com banco**
O Sistema de Funcionários da UC 1, agora com os dados no PostgreSQL

04 **Prática guiada: Loja Conecta**
Produtos, pedidos com RETURNING e a transação que grava tudo ou nada

★ **Atividade final**
Um tema sorteado para cada um: do briefing ao sistema funcionando

NESTE MATERIAL

Exemplo explicado
Um caso resolvido passo a passo.

Prática
Individual e digital, no seu computador e no seu banco na Aiven, com o professor acompanhando.

Terminal
Os programas foram testados: a saída mostrada é a que aparece na tela.

ONDE ESTAMOS

O que muda neste capítulo

NOS CAPÍTULOS 3 E 4

Quem falava com o banco era você

Cada comando era digitado na Query Tool do pgAdmin, conectado à Aiven.

NESTE CAPÍTULO • SEÇÃO 01

Um **programa Python** conecta na Aiven e roda os comandos, com a senha guardada fora do código.

NA UC 1

O CRUD guardava tudo numa lista

```
funcionarios.append(novo_funcionario)
```

E salvava num `dados.json`, só na sua máquina.

NESTE CAPÍTULO • SEÇÕES 02 E 03

O mesmo CRUD, com os dados no **PostgreSQL**.

NO CAPÍTULO 4

FK, CHECK e GENERATED protegiam o banco

O erro aparecia em vermelho, na aba Messages.

NESTE CAPÍTULO • SEÇÃO 04

Cada regra vira uma **mensagem** no programa, e o pedido é gravado inteiro ou não é gravado.

No fim do capítulo, cada um recebe um tema por sorteio e vai do briefing ao sistema funcionando: modelagem, banco na Aiven e CRUD em Python.

SEÇÃO 01

Python conversa com o banco

01

O arquivo .env, o psycopg e o primeiro script: apagar, criar e preencher o banco.

01 • Conexão e .env | 02 • Classe e parâmetros | 03 • CRUD | 04 • Loja Conecta

01 • CONEXÃO

Do pgAdmin ao programa

Num sistema de verdade, ninguém digita SQL. O usuário escolhe uma opção no menu, e o programa monta o comando, envia ao banco e mostra o resultado.

Usuário
escolhe no menu e digita os dados

→

Programa Python
monta o SQL com os dados digitados

→

psycopg
leva o comando pela internet e traz a resposta

→

PostgreSQL na Aiven
confere as regras e guarda os dados

Os dados ficam

Fechou o programa, desligou o PC: os dados continuam no servidor. O `dados.json` da UC 1 morava só na sua máquina.

Vários lugares, um banco

O mesmo banco atende o laboratório, a sua casa e, um dia, o app do celular.

As regras protegem

As FKs e os CHECKs do Capítulo 4 recusam o dado errado, venha do pgAdmin ou do Python.

psycopg é a biblioteca que faz a ponte entre o Python e o PostgreSQL: o programa fala Python, o banco fala SQL, e o psycopg leva e traz.

O que instalar

No terminal do VS Code, uma vez só por computador:

```
TERMINAL
> pip install "psycopg[binary]" python-dotenv
...
Successfully installed psycopg-3.3.6 psycopg-binary-3.3.6
python-dotenv-1.1.1
```

PACOTE	PARA QUÊ
`psycopg`	A ponte entre o Python e o PostgreSQL. No código: `import psycopg`.
`[binary]`	Vem pronto para o Windows: não precisa instalar mais nada.
`python-dotenv`	Lê o arquivo `.env`, onde fica a senha. No código: `from dotenv import load_dotenv`.

CONFERIR

```
TERMINAL
> python -c "import psycopg;
print(psycopg.__version__)"
3.3.6
```

Apareceu um número? Está instalado. `ModuleNotFoundError`? O pip instalou num Python diferente do que o VS Code usa: rode o pip no terminal do próprio VS Code.

Na internet você vai ver muito **psycopg2**, a versão anterior. A ideia é a mesma; neste capítulo usamos a versão 3: `import psycopg`.

01 • CONEXÃO • AIVEN

Os dados da conexão estão na Aiven

Os mesmos do Capítulo 3, quando o pgAdmin conectou na Aiven. Em **console.aiven.io** → seu serviço → **Overview** → **Connection information**.

NA AIVEN	NO PGADMIN (CAPÍTULO 3)	NO PYTHON	EXEMPLO
Host	Host name/address	`host`	`pg-escola-turma.a.aivencloud.com`
Port	Port	`port`	`12691` (a do seu serviço)
Database name	Maintenance database	`dbname`	`rh` (o banco que você criou)
User	Username	`user`	`avnadmin`
Password	Password	`password`	a do console da Aiven
SSL mode	SSL mode	`sslmode`	`require`

require: a conversa com a Aiven vai criptografada. Sem ele, a Aiven recusa a conexão.

A senha é sua. Quem tem esses seis dados entra no seu banco: lê, altera e apaga tudo.

Por que não escrever a senha no código

ASSIM NÃO

setup_rh.py

```
conexao = psycopg.connect(
    host="pg-escola-turma.a.aivencloud.com",
    port=12691,
    dbname="rh",
    user="avnadmin",
    password="AVNS_x8Kq2pW7rT",    # a senha no código
    sslmode="require",
)
```

`git push` → `github.com/você/projeto` → `qualquer pessoa lê`

- O código vai para o **GitHub**. A senha não pode ir junto.
- Existem robôs que varrem o GitHub atrás de senhas e chaves publicadas.
- Cada aluno tem a sua senha: com ela no código, o colega precisa editar o programa para rodar.
- Trocou a senha na Aiven? Com a senha num arquivo à parte, muda um arquivo só, e o código fica igual.

A solução: o arquivo .env

Os dados da conexão ficam num arquivo separado, que **não** vai para o GitHub. O código só lê esse arquivo.

⚠ Senha que foi para o GitHub é senha vazada: mesmo apagando depois, ela fica no histórico dos commits. Troque a senha no console da Aiven.

01 • .ENV • SINTAXE

O arquivo .env

.env

```
# Conexão com a Aiven (não vai para o GitHub)
DB_HOST=pg-escola-turma.a.aivencloud.com
DB_PORT=12691
DB_NAME=rh
DB_USER=avnadmin
DB_PASSWORD=AVNS_sua_senha_aqui
DB_SSLMODE=require
```

PARECE UM DICIONÁRIO

```
{"DB_HOST": "pg-escola-turma.a.aivencloud.com", "DB_PORT": "12691", ...}
```

Uma chave, um valor. E todo valor chega no Python como **texto**: até a porta é `"12691"`.

- Uma informação por linha: `NOME=valor`.
- Sem espaço em volta do `=` e sem aspas: o formato mais simples, que toda ferramenta aceita.
- Nomes em MAIÚSCULAS, por costume. O `DB_` lembra que é do banco.
- `#` no começo da linha: comentário.
- O nome do arquivo é só `.env`: nada antes do ponto, nada de `.txt` depois.
- Fica na **mesma pasta** do programa.

Crie pelo VS Code: **New File** → `.env`. O Bloco de Notas costuma salvar como `.env.txt`.

O .env fica fora do GitHub

```
rh/
├── .env            sua senha · não vai para o GitHub
├── .env.exemplo    o modelo, sem senha · vai
├── .gitignore      a lista do que o git ignora
├── setup_rh.py
└── testar_conexao.py
```

.gitignore

```
.env
```

.env.exemplo

```
DB_HOST=seu-servico.a.aivencloud.com
DB_PORT=12345
DB_NAME=rh
DB_USER=avnadmin
DB_PASSWORD=cole_a_senha_aqui
DB_SSLMODE=require
```

1. Crie o `.gitignore` com a linha `.env` **antes** do primeiro commit. O git passa a fingir que o arquivo não existe.
2. Suba o `.env.exemplo`: quem baixar o projeto copia, renomeia para `.env` e coloca os próprios dados.
3. Antes de cada commit, confira o `git status`: o `.env` não pode aparecer na lista.

TERMINAL

```
> git status
Untracked files:
        .env.exemplo
        .gitignore
        setup_rh.py
(o .env não aparece: está sendo ignorado)
```

No repositório da disciplina, o `.gitignore` já tem a linha `.env`.

Lendo o .env no programa

```python
import os
from dotenv import load_dotenv

load_dotenv()                       # procura o .env e carrega os valores

servidor = os.getenv("DB_HOST")     # 'pg-escola-turma.a.aivencloud.com'
porta = os.getenv("DB_PORT")        # '12691'  (texto)
senha = os.getenv("DB_SENHA")       # None: esse nome não existe no .env
```

Compare com o dicionário: `os.getenv("DB_HOST")` funciona como `dados.get("DB_HOST")`. Chave que não existe devolve `None`, sem erro.

COMANDO	O QUE FAZ
`import os`	Biblioteca do próprio Python para falar com o sistema operacional.
`load_dotenv()`	Procura o `.env` na pasta do programa (e, se não achar, nas pastas de cima) e carrega cada linha no **ambiente**.
`os.getenv("NOME")`	Lê um valor do ambiente. Sempre texto, ou `None`.

Ambiente são variáveis que o sistema operacional entrega a cada programa. O `load_dotenv` copia o `.env` para lá, e o `os.getenv` lê de lá.

As peças do psycopg

Cada coisa que você fazia no pgAdmin tem uma linha equivalente no Python.

NO PGADMIN	NO PYTHON, COM PSYCOPG	O QUE É
Conectar no servidor, com a senha	`conexao = psycopg.connect(host=..., ...)`	A **conexão**: a ligação com o servidor.
Abrir a Query Tool	`cursor = conexao.cursor()`	O **cursor**: quem roda os comandos e guarda a resposta.
Escrever o SQL e apertar F5	`cursor.execute("SELECT ...")`	O SQL vai como texto, igual ao da Query Tool.
Ver a grade do resultado	`cursor.fetchone()` • `cursor.fetchall()`	Uma linha • todas as linhas.
Nada: salva sozinho a cada comando	`conexao.commit()`	Grava de vez o que foi feito.
Fechar a aba e desconectar	`conexao.close()`	Libera a conexão no servidor.

A ordem é sempre essa: **conectar** → **cursor** → **execute** → **(fetch ou commit)** → **fechar**. O resto do capítulo é variação desse molde.

EXEMPLO EXPLICADO • PRIMEIRO PROGRAMA

testar_conexao.py: o Python chegou na Aiven?

testar_conexao.py

```python
import os
import psycopg
from dotenv import load_dotenv

load_dotenv()

if os.getenv("DB_HOST") is None:
    print("Não encontrei o .env. Confira o nome do arquivo e a pasta.")
else:
    print("Conectando em", os.getenv("DB_HOST"))

    conexao = psycopg.connect(
        host=os.getenv("DB_HOST"),
        port=os.getenv("DB_PORT"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD"),
        sslmode=os.getenv("DB_SSLMODE"),
    )
    cursor = conexao.cursor()

    cursor.execute("SELECT current_database(), CURRENT_DATE")
    linha = cursor.fetchone()
    print("Conectado! Resposta do banco:", linha)

    conexao.close()
```

TERMINAL

```
> python testar_conexao.py
Conectando em pg-escola-turma.a.aivencloud.com
Conectado! Resposta do banco: ('rh', datetime.date(2026,
10, 29))
```

- Antes de conectar, o programa confere se o `.env` foi lido: sem ele, avisa em vez de tentar.
- Cada argumento do `connect` vem de uma linha do `.env`.
- `current_database()` responde o nome do banco: confirma que você caiu no banco certo.
- O `fetchone()` devolve **uma linha**, como tupla: uma posição por coluna do SELECT.
- A data veio como `datetime.date`: o psycopg converte os tipos (Seção 02).

Antes de rodar: crie o banco **rh** na Aiven (pgAdmin → Databases → Create → Database) e o `.env` com `DB_NAME=rh`.

EXEMPLO EXPLICADO • .ENV • ERROS COMUNS

Quando o .env não é lido

SITUAÇÃO	O QUE APARECE	COMO RESOLVER
O arquivo ficou `.env.txt`	`Não encontrei o .env. Confira o nome do arquivo e a pasta.`	Renomeie pelo VS Code. No Explorer do Windows, ligue **Exibir → Extensões de nomes de arquivos** para ver o `.txt`.
O `.env` está em outra pasta	`Não encontrei o .env. Confira o nome do arquivo e a pasta.`	Deixe o `.env` na pasta do programa.
Faltou o `load_dotenv()`	`connection to server at "localhost" (::1), port 5432 failed: fe_sendauth: no password supplied`	Todo `os.getenv` deu `None`, e o psycopg tentou um PostgreSQL **no próprio computador**. Sem PostgreSQL instalado, aparece *Connection refused*.
`DB_PASS` no .env, `DB_PASSWORD` no código	`... failed: fe_sendauth: no password supplied`	O nome precisa ser **igual** nos dois lugares, letra por letra.

Na dúvida, imprima o que foi lido: `print(os.getenv("DB_HOST"))`. Saiu `None`? O problema está no `.env`, não no banco. Só não imprima a senha.

01 • A SITUAÇÃO DA EXPOSIÇÃO

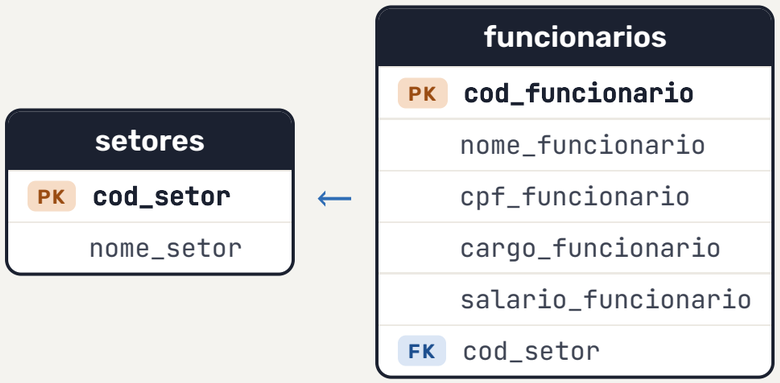
O Sistema RH ganha um banco

O Sistema de Funcionários da UC 1 guardava cada funcionário num dicionário. Agora cada dicionário vira uma **linha**, e cada chave, uma **coluna**.

NA UC 1

```
novo_funcionario = {
    "nome": nome,
    "cpf": cpf,
    "cargo": cargo,
    "salario": salario
}
funcionarios.append(novo_funcionario)
```

NESTE CAPÍTULO • BANCO RH



REGRA	NA UC 1	NO BANCO
CPF não se repete	`cpf_exist(cpf)`	`UNIQUE`
CPF com 11 dígitos	`len(cpf) != 11`	`ck_cpf_tamanho`
Salário maior que zero	`salario <= 0`	`ck_salario_positivo`
Setor existe	não havia	`fk_func_setor`
Código do funcionário	a posição na lista	`GENERATED ALWAYS`

As regras da UC 1, agora também no banco: valem para qualquer programa que use o banco rh.

setup_rh.py: conectar e apagar

setup_rh.py

```
import os
import psycopg
from dotenv import load_dotenv

load_dotenv()

conexao = psycopg.connect(
    host=os.getenv("DB_HOST"),
    port=os.getenv("DB_PORT"),
    dbname=os.getenv("DB_NAME"),
    user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASSWORD"),
    sslmode=os.getenv("DB_SSLMODE"),
)
cursor = conexao.cursor()

# 1) Apagar: filhos antes dos pais
cursor.execute("DROP TABLE IF EXISTS funcionarios")
cursor.execute("DROP TABLE IF EXISTS setores")
```

O mesmo roteiro dos scripts `.sql` do Capítulo 4: apagar, criar e preencher. Cada comando vira um `cursor.execute(...)`, um depois do outro.

- O SQL vai entre aspas: para o Python, é só um **texto**. Quem entende o SQL é o servidor.
- O `;` no fim é opcional: cada `execute` leva um comando.
- A ordem do DROP é a mesma do Capítulo 4: **filhos antes dos pais**. Apagar `setores` primeiro esbarra na FK.
- O `IF EXISTS` deixa o script rodar de novo: na segunda vez as tabelas já existem.

setup_rh.py: criar as tabelas

```
# 2) Criar: pais antes dos filhos
cursor.execute("""
    CREATE TABLE setores (
      cod_setor  integer     GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      nome_setor varchar(60) NOT NULL UNIQUE
    )
""")
cursor.execute("""
    CREATE TABLE funcionarios (
      cod_funcionario     integer       GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
      nome_funcionario    varchar(100)  NOT NULL,
      cpf_funcionario     char(11)      NOT NULL UNIQUE,
      cargo_funcionario   varchar(60)   NOT NULL,
      salario_funcionario numeric(10,2) NOT NULL,
      cod_setor           integer       NOT NULL,
      CONSTRAINT ck_cpf_tamanho      CHECK (LENGTH(cpf_funcionario) = 11),
      CONSTRAINT ck_salario_positivo CHECK (salario_funcionario > 0),
      CONSTRAINT fk_func_setor FOREIGN KEY (cod_setor) REFERENCES setores (cod_setor)
    )
""")
print("Tabelas criadas.")
```

- As **aspas triplas** `"""` guardam um texto de várias linhas, como no menu da UC 1.
- Dentro delas vai o `CREATE TABLE` do Capítulo 4, sem mudar nada.
- Pais antes dos filhos: `setores` antes de `funcionarios`.
- Toda regra com nome: `ck_cpf_tamanho`, `ck_salario_positivo`, `fk_func_setor`. Na Seção 03, o nome vira mensagem.

Dica: escreva e teste o `CREATE` no pgAdmin primeiro. Funcionou lá? Cole entre as aspas triplas.

setup_rh.py: preencher e gravar

```
# 3) Dados: pais antes dos filhos
cursor.execute("""
    INSERT INTO setores (nome_setor)
    VALUES ('Administrativo'), ('Comercial'), ('Tecnologia')
""")
cursor.execute("""
    INSERT INTO funcionarios
      (nome_funcionario, cpf_funcionario, cargo_funcionario, salario_funcionario, cod_setor)
    VALUES ('Ana Souza',     '12345678901', 'Analista de RH',    4200.00, 1),
           ('Bruno Lima',    '23456789012', 'Vendedor',          3100.00, 2),
           ('Carla Mendes',  '34567890123', 'Desenvolvedora',    6800.00, 3),
           ('Diego Rocha',   '45678901234', 'Suporte técnico',   3500.00, 3),
           ('Elisa Martins', '56789012345', 'Gerente comercial', 7500.00, 2)
""")
print("Dados inseridos.")

# 4) Gravar tudo e fechar
conexao.commit()
print("Banco rh pronto!")
conexao.close()
```

- Os códigos não aparecem no INSERT: o `GENERATED` numera 1, 2, 3… na ordem das linhas.
- Por isso `cod_setor` 1 é Administrativo, 2 é Comercial e 3 é Tecnologia.
- `commit()` grava de vez tudo o que o script fez. O próximo slide mostra por que ele é obrigatório.
- `close()` desliga do servidor: a conexão não fica presa.

EXEMPLO EXPLICADO • RODANDO O SETUP

O banco rh montado pelo Python

TERMINAL

```
> python setup_rh.py
Tabelas criadas.
Dados inseridos.
Banco rh pronto!
```

- Rode de novo: o resultado é o mesmo. O DROP apaga, o CREATE recria e o INSERT preenche: o banco **sempre começa do zero**.
- Um colega baixa o projeto, cria o próprio `.env` e roda o mesmo script: tem o mesmo banco, na Aiven dele.

CONFIRA NO PGADMIN

Botão direito em **Tables** → **Refresh**. Na Query Tool do banco rh:

```
SELECT * FROM funcionarios;
```

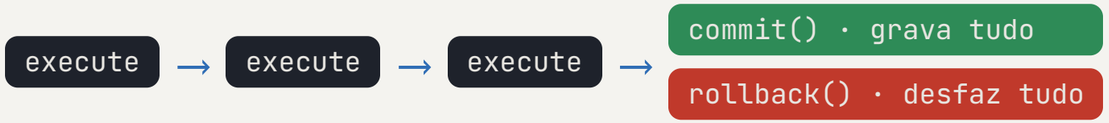
cod_funcionario	nome_funcionario	cargo_funcionario	salario_funcionario	cod_setor
1	Ana Souza	Analista de RH	4200.00	1
2	Bruno Lima	Vendedor	3100.00	2
3	Carla Mendes	Desenvolvedora	6800.00	3
4	Diego Rocha	Suporte técnico	3500.00	3
5	Elisa Martins	Gerente comercial	7500.00	2

5 linhas · sem a coluna cpf_funcionario, para caber

01 • TRANSAÇÃO

Sem commit, nada fica gravado

No primeiro `execute`, o psycopg abre uma **transação**: tudo o que vem depois fica pendente, esperando uma decisão.



- `close()` sem `commit()` desfaz o que estava pendente.
- No pgAdmin você nunca precisou: a Query Tool vem com o **auto-commit** ligado, e cada comando é gravado na hora.
- No PostgreSQL até o `DROP` e o `CREATE` entram na transação e podem ser desfeitos.

TESTE: O MESMO SCRIPT, SEM A LINHA DO COMMIT

```
TERMINAL
> python setup_rh.py
Tabelas criadas.
Dados inseridos.
Banco rh pronto!
```

O programa diz que deu certo, mas no pgAdmin, depois do Refresh, **não há nenhuma tabela**. Nenhum erro, nenhum aviso: o erro mais traiçoeiro do capítulo.

Tudo ou nada: é isso que a Seção 04 usa para gravar um pedido com todos os itens, ou não gravar nada.

E se der errado?

O mesmo `setup_rh.py`, com a senha errada no `.env`:

TERMINAL

```
> python setup_rh.py
Traceback (most recent call last):
  File "D:\projetos\rh\setup_rh.py", line 7, in <module>
    conexao = psycopg.connect(
  File "...\site-packages\psycopg\connection.py", line 134, in connect
    raise new_ex.with_traceback(None)
psycopg.OperationalError: connection failed: connection to server at "34.39.x.x", port 12691 failed:
FATAL:  password authentication failed for user "avnadmin"
```

O programa morre

Nenhuma linha depois do erro roda. Num menu, o usuário perde o programa inteiro por causa de uma opção.

A mensagem assusta

Quem usa o sistema não sabe o que é *Traceback*. Bastava dizer: “não foi possível conectar”.

A conexão pode ficar aberta

Se o erro acontece **depois** de conectar, o `close()` nunca roda. A conexão fica presa no servidor, segurando as tabelas.

Na UC 1, o `try/except ValueError` salvou o `int(input())`. Com banco, a ideia é a mesma, com uma peça nova: o **finally**.

try, except e finally: o molde

```python
conexao = None

try:
    # o caminho normal: pode dar errado em qualquer linha
    conexao = psycopg.connect(...)
    cursor = conexao.cursor()
    cursor.execute(...)
    conexao.commit()

except psycopg.OperationalError as erro:
    # não conseguiu conectar
    print("Não foi possível conectar ao banco.")

except psycopg.Error as erro:
    # conectou, mas um comando falhou
    print("Erro no SQL. Nada foi gravado.")
    conexao.rollback()

finally:
    # roda SEMPRE: com erro ou sem erro
    if conexao is not None:
        conexao.close()
```

BLOCO	QUANDO RODA
`try`	Sempre começa. Parou no primeiro erro.
`except Tipo`	Só se acontecer um erro daquele tipo. O Python testa de cima para baixo e usa o **primeiro** que servir.
`as erro`	Guarda o erro numa variável: `print(erro)` mostra a mensagem do banco.
`finally`	**Sempre**, no fim: deu certo, deu erro ou saiu por um `return`. O lugar de fechar a conexão.



O específico vem antes do geral. `OperationalError` é um tipo de `psycopg.Error`: com a ordem trocada, o geral pegaria tudo.

O mesmo script, três finais

TUDO CERTO

`try`: conecta, apaga, cria, insere

↓

`commit()`: grava tudo

↓

nenhum **`except`** roda

↓

`finally`: fecha a conexão

NÃO CONECTOU

`try`: o **`connect`** falha na 1ª linha

↓

`except OperationalError`: avisa o usuário

↓

`finally`: **`conexao`** ainda é **`None`**, não há o que fechar

UM COMANDO FALHOU

`try`: conecta, apaga… e o CREATE falha

↓

`except psycopg.Error`: **`rollback()`** desfaz até o DROP

↓

`finally`: fecha a conexão

Por que `conexao = None` antes do `try`? No caminho do meio, o `connect` falhou e a variável nunca recebeu valor. Sem a linha do `None`, o `finally` quebraria com `NameError: name 'conexao' is not defined`.

PENSE E RESPONDA No caminho da direita, o DROP já tinha rodado quando o CREATE falhou. Depois do rollback, as tabelas antigas existem ou não?

setup_rh.py com try, except e finally

setup_rh.py

```python
# ... os imports e o load_dotenv(), como antes

conexao = None   # ainda não há conexão: o finally precisa saber disso

try:
    conexao = psycopg.connect(...)   # os 6 dados do .env, como antes
    cursor = conexao.cursor()

    cursor.execute("DROP TABLE IF EXISTS funcionarios")
    cursor.execute("DROP TABLE IF EXISTS setores")
    # ... os CREATE e os INSERT, como antes

    conexao.commit()
    print("Banco rh pronto!")

except psycopg.OperationalError as erro:
    print("Não foi possível conectar ao banco.")
    print(erro)

except psycopg.Error as erro:
    print("Erro no SQL. Nada foi gravado.")
    print(erro)
    conexao.rollback()

finally:
    if conexao is not None:
        conexao.close()
        print("Conexão fechada.")
```

- Tudo o que fala com o banco foi para dentro do `try`, com mais 4 espaços de recuo.
- O `print(erro)` mostra a mensagem do banco: ajuda você a achar o problema.
- O `close()` saiu do fim do `try` e foi para o `finally`: agora ele roda em qualquer caso.

O arquivo completo está no repositório: `Capitulo 5 - Python e PostgreSQL/rh/setup_rh.py`.

EXEMPLO EXPLICADO • TRY, EXCEPT E FINALLY

Três execuções do mesmo script

1 • TUDO CERTO

```
TERMINAL
> python setup_rh.py
Tabelas criadas.
Dados inseridos.
Banco rh pronto!
Conexão fechada.
```

O `try` foi até o fim e o `finally` fechou.

2 • SENHA ERRADA NO .ENV

```
TERMINAL
> python setup_rh.py
Não foi possível conectar ao banco.
connection failed: connection to
server at "34.39.x.x", port 12691
failed: FATAL:  password
authentication failed for user
"avnadmin"
```

Caiu no `OperationalError`. Não aparece *Conexão fechada*: ela nunca abriu.

3 • CREAT NO LUGAR DE CREATE

```
TERMINAL
> python setup_rh.py
Erro no SQL. Nada foi gravado.
syntax error at or near "CREAT"
LINE 2:         CREAT TABLE
funcionarios (
                ^
Conexão fechada.
```

Caiu no `psycopg.Error`. O `rollback()` desfez até o DROP: **as tabelas antigas continuam lá**, com os 5 funcionários.

OUTRAS MENSAGENS DE CONEXÃO

```
FATAL:  database "rhh" does not exist
```

o `DB_NAME` não bate com o banco criado na Aiven

```
failed to resolve host 'pg-escola-turmaa.a.aivencloud.com': [Errno 11001] getaddrinfo failed
```

o `DB_HOST` tem erro de digitação, ou o computador está sem internet

Os tipos de erro do psycopg

```
psycopg.Error                pega todos os erros do banco
├── OperationalError          não conectou: senha, host, banco, internet
├── ProgrammingError          o SQL está errado
│   ├── errors.SyntaxError        CREAT TABLE
│   ├── errors.UndefinedTable     tabela que não existe
│   └── errors.UndefinedColumn    coluna que não existe
├── IntegrityError            uma regra do banco recusou
│   ├── errors.UniqueViolation      UNIQUE
│   ├── errors.ForeignKeyViolation  FOREIGN KEY
│   ├── errors.CheckViolation       CHECK
│   └── errors.NotNullViolation     NOT NULL
└── DataError                 o valor não serve: texto grande demais, "abc" num integer
```

- Cada regra do Capítulo 4 vira um **tipo de erro** no Python.
- Os de baixo são casos dos de cima: um `CheckViolation` também é `IntegrityError` e também é `psycopg.Error`.
- No `except`, escreva com o caminho: `psycopg.errors.UniqueViolation`.

Na Seção 03, cada tipo vira uma mensagem para o usuário: "CPF já cadastrado", "esse setor não existe".

Por que fechar a conexão importa

O servidor tem limite

Cada conexão aberta ocupa memória e uma vaga no servidor. O PostgreSQL aceita um número limitado de conexões ao mesmo tempo (o padrão é 100), e o plano gratuito da Aiven é um servidor pequeno.

Transação aberta segura as tabelas

Uma conexão que fez um `DROP` ou um `UPDATE` e não terminou segura aquela tabela. No pgAdmin, o `SELECT` fica “rodando” sem resposta até a conexão fechar.

Programa que não termina

O `setup` acaba em segundos, e o sistema fecha tudo ao sair. Um **menu** fica aberto a aula inteira: cada erro sem `close()` deixa uma conexão para trás.

O finally não esquece

Ele roda com erro, sem erro e até depois de um `return`. Com o `close()` lá, não importa como o `try` terminou.

Regra do capítulo: **abriu, fecha no finally**. Na Seção 02, a classe Banco faz isso por você em toda consulta.

PRÁTICA 1 • 20 MIN • INDIVIDUAL

O setup do Cinema Estrela

O Cinema Estrela quer um banco para os filmes e as sessões. Na Aiven, crie o banco **cinema** e escreva o `setup_cinema.py`, no molde do `setup_rh.py`.

TABELA • COLUNA	REGRA
`filmes.cod_filme`	gerado pelo banco (GENERATED ALWAYS)
`filmes.titulo_filme`	texto até 100, obrigatório
`filmes.duracao_filme`	inteiro, em minutos, obrigatório • maior que zero: `ck_duracao`
`filmes.classificacao_filme`	texto até 2, obrigatório • só L, 10, 12, 14, 16 ou 18: `ck_classificacao`
`sessoes.cod_sessao`	gerado pelo banco
`sessoes.cod_filme`	obrigatória • FK para filmes: `fk_sessao_filme`
`sessoes.data_sessao` • `horario_sessao`	`date` e `time`, obrigatórios
`sessoes.sala_sessao`	inteiro, obrigatório • de 1 a 5: `ck_sala`

1. Pasta **cinema** com `.env` (`DB_NAME=cinema`), `.env.exemplo` e `.gitignore`.
2. No `setup_cinema.py`: DROP das duas tabelas, CREATE com as regras e INSERT dos dados do próximo slide, tudo com `try/except/finally`.
3. Rode **duas vezes**: as duas terminam em *Banco cinema pronto!* e *Conexão fechada.*
4. Confira no pgAdmin: 5 filmes e 6 sessões.

PRÁTICA 1 • DADOS PARA LANÇAR E TESTES

O setup do Cinema Estrela

FILMES • NA ORDEM: O BANCO GERA 1 A 5

titulo_filme	duracao	classif.
O Último Farol	112	12
Corrida nas Dunas	98	L
Código Vermelho	125	16
A Ilha dos Gatos	90	L
Noite sem Fim	105	14

SESSOES • NA ORDEM: O BANCO GERA 1 A 6

cod_filme	data_sessao	horario	sala
1	2026-11-06	19:00	1
1	2026-11-06	21:30	1
2	2026-11-07	15:00	2
3	2026-11-07	20:00	3
4	2026-11-08	14:00	2
2	2026-11-08	16:30	4

PROVOQUE DOIS ERROS, UM DE CADA VEZ

1. Troque uma letra do `DB_PASSWORD` no `.env`. Rode. Qual `except` pegou? Apareceu *Conexão fechada.*? Desfaça.
2. Escreva `CREAT` no CREATE de `sessoes`. Rode. Qual `except` pegou? No pgAdmin, os filmes da rodada anterior continuam lá? Desfaça.

Anote as duas mensagens como comentário no fim do `setup_cinema.py`.

PENSE E RESPONDA No erro 2, o DROP de sessoes e o DROP de filmes rodaram antes do CREAT. Por que os filmes continuam no banco?

SEÇÃO 02

Uma classe para o banco

02

A conexão num lugar só e as consultas com parâmetros: o valor que o usuário digita entra no SQL com segurança.

Repetir a conexão em todo lugar?

Para rodar **um** SELECT com segurança, o setup precisou de tudo isto:

```
conexao = None
try:
    conexao = psycopg.connect(host=..., port=..., dbname=...,
                              user=..., password=..., sslmode=...)
    cursor = conexao.cursor()
    cursor.execute("SELECT ...")      # a única linha que muda
    linhas = cursor.fetchall()
finally:
    if conexao is not None:
        conexao.close()
```

O CRUD tem uma consulta em cada função: cadastrar, ver, alterar, remover, buscar… Copiar esse bloco em todas é pedir para esquecer um `close()`.

A SOLUÇÃO: UMA CLASSE BANCO

Como a classe `Conta` da POO: os **atributos** guardam os dados da conexão, e os **métodos** fazem o trabalho.



MÉTODO	PARA	DEVOLVE
`consultar(sql, parametros)`	SELECT	a lista de linhas
`executar(sql, parametros)`	INSERT, UPDATE, DELETE	quantas linhas mudaram

classBanco.py: os dados da conexão

classBanco.py

```python
import os
import psycopg
from dotenv import load_dotenv

load_dotenv()


class Banco():
    def __init__(self):
        self.servidor = os.getenv("DB_HOST")
        self.porta = os.getenv("DB_PORT")
        self.nome_banco = os.getenv("DB_NAME")
        self.usuario = os.getenv("DB_USER")
        self.senha = os.getenv("DB_PASSWORD")
        self.ssl = os.getenv("DB_SSLMODE")

    def conectar(self):
        return psycopg.connect(
            host=self.servidor,
            port=self.porta,
            dbname=self.nome_banco,
            user=self.usuario,
            password=self.senha,
            sslmode=self.ssl,
        )
```

- O nome do arquivo segue o padrão da POO: `classBanco.py`, e no programa `from classBanco import Banco`.
- O `load_dotenv()` roda uma vez, quando o arquivo é importado.
- O `__init__` copia o `.env` para os atributos: `Banco()` não precisa de nenhum argumento.
- `conectar()` abre uma conexão nova e devolve. Quem pediu é quem fecha.

```
rh/
├── .env
├── classBanco.py   a classe
├── setup_rh.py
└── main.py         usa a classe
```

classBanco.py: consultar e executar

```
    # SELECT: devolve a lista de linhas (cada linha é uma tupla)
    def consultar(self, sql, parametros=None):
        conexao = self.conectar()
        try:
            cursor = conexao.cursor()
            cursor.execute(sql, parametros)
            return cursor.fetchall()
        finally:
            conexao.close()

    # INSERT, UPDATE ou DELETE: grava e devolve quantas linhas mudaram
    def executar(self, sql, parametros=None):
        conexao = self.conectar()
        try:
            cursor = conexao.cursor()
            cursor.execute(sql, parametros)
            conexao.commit()
            return cursor.rowcount
        finally:
            conexao.close()
```

- Os dois métodos ficam **dentro** da classe, com o mesmo recuo do `conectar`.
- `parametros=None`: o argumento é opcional. Um SELECT sem valores digitados não precisa dele.
- Só o `executar` tem `commit()`: o SELECT não muda nada.
- `rowcount`: quantas linhas o comando inseriu, alterou ou apagou.

EXEMPLO EXPLICADO • A CLASSE POR DENTRO

Por que o consultar() não tem except?

```python
def consultar(self, sql, parametros=None):
    conexao = self.conectar()       # 1
    try:
        cursor = conexao.cursor()
        cursor.execute(sql, parametros)
        return cursor.fetchall()    # 2
    finally:
        conexao.close()             # 3
```

1. O `conectar()` fica **fora** do try: se ele falha, não há conexão para fechar, e o erro sobe direto para quem chamou.
2. O `return` não pula o `finally`: o Python fecha a conexão e só depois devolve as linhas.
3. Deu erro no SQL? O `finally` fecha, e o erro **continua subindo**.

TESTE: O FINALLY RODA ANTES DO RETURN

```python
def teste():
    try:
        return "valor"
    finally:
        print("finally rodou")

print(teste())
```

TERMINAL

```
> python teste.py
finally rodou
valor
```

Divisão de tarefas: a **classe** garante que a conexão fecha. **Quem chamou** decide o que dizer ao usuário, com o próprio `except`: “CPF já cadastrado” só faz sentido no cadastro.

Usando a classe

```python
from classBanco import Banco

banco = Banco()      # lê o .env uma vez

setores = banco.consultar(
    "SELECT cod_setor, nome_setor FROM setores ORDER BY cod_setor")
print(setores)

for cod, nome in setores:
    print(cod, "-", nome)

apagados = banco.executar(
    "DELETE FROM funcionarios WHERE cod_funcionario = 99")
print(apagados, "linha(s) apagada(s)")
```

TERMINAL

```
> python teste_classe.py
[(1, 'Administrativo'), (2, 'Comercial'), (3, 'Tecnologia')]
1 - Administrativo
2 - Comercial
3 - Tecnologia
0 linha(s) apagada(s)
```

- Uma linha por consulta: o resto (conectar, cursor, fechar) fica escondido na classe.
- `for cod, nome in setores` desempacota cada tupla: uma variável por coluna, na ordem do SELECT.
- O DELETE do funcionário 99 não deu erro: não havia ninguém com esse código, e o `rowcount` foi **0**.

Cada chamada abre e fecha a própria conexão. Para consultas **pontuais**, uma de cada vez, é o jeito mais seguro.

Linhas, tuplas e tipos

```
linhas = banco.consultar("""
    SELECT cod_funcionario, nome_funcionario, salario_funcionario
    FROM funcionarios WHERE cod_setor = 3""")
```

EXPRESSÃO	VALOR
`linhas`	`[(3, 'Carla Mendes', Decimal('6800.00')), (4, 'Diego Rocha', Decimal('3500.00'))]`
`linhas[0]`	`(3, 'Carla Mendes', Decimal('6800.00'))`
`linhas[0][1]`	`'Carla Mendes'`
`len(linhas)`	2 · sem resultado: `[]`, e `len` dá 0

Na UC 1: `funcionario["nome"]`. Com tupla: `linha[1]`, pela posição da coluna no SELECT.

NO POSTGRESQL	NO PYTHON	EXEMPLO
`integer`	`int`	`3`
`numeric(10,2)`	`Decimal`	`Decimal('4200.50')`
`varchar` • `char` • `text`	`str`	`'Carla Mendes'`
`date`	`datetime.date`	`date(2026, 10, 29)`
`time`	`datetime.time`	`time(19, 0)`
`boolean`	`bool`	`True`
`NULL`	`None`	`None`

- `Decimal` guarda dinheiro sem erro de arredondamento. A f-string formata igual: `f"R$ {salario:,.2f}"`.
- Data no formato brasileiro: `f"{data:%d/%m/%Y}"` → 29/10/2026. Hora: `f"{horario:%H:%M}"` → 19:00.

O valor vem do usuário

O SQL quase nunca é fixo: o setor, o nome e o salário são **digitados**.

```
cod_setor = int(input("Ver os funcionários de qual setor? "))
```

A primeira ideia é colar o valor no texto com f-string:

```
banco.consultar(f"SELECT ... WHERE cod_setor = {cod_setor}")
```

Funciona com número. Com texto, começam os problemas:

- Texto no SQL precisa de aspas simples, e esquecer dá erro.
- Um nome como **Joana D'Ávila** tem um apóstrofo, que fecha a aspa antes da hora.
- Quem digita pode **reescrever o seu SQL**. Isso tem nome: SQL injection.

O jeito certo: parâmetros

```
banco.consultar(
    "SELECT ... WHERE cod_setor = %s",
    (cod_setor,),
)
```

O `%s` marca o lugar do valor. O valor vai separado, numa tupla, e o psycopg entrega os dois ao servidor sem misturar.

Parâmetros: o molde

```
banco.consultar(
    "SELECT colunas FROM tabela WHERE coluna1 = %s AND coluna2 = %s",
    (valor1, valor2),
)

banco.executar(
    "UPDATE tabela SET coluna = %s WHERE cod = %s",
    (novo_valor, cod),
)
```

1º `%s` ← 1º valor da tupla 2º `%s` ← 2º valor e assim por diante

- `%s` serve para **qualquer tipo**: número, texto, data, `None`.
- **Sem aspas** em volta do `%s`, mesmo para texto: o psycopg cuida disso.
- Os valores vão numa tupla, **na ordem** dos `%s`. Quantos `%s`, tantos valores.
- Um valor só? A vírgula é obrigatória: `(cod,)`. Sem ela, `(cod)` é só um número entre parênteses. Também vale uma lista: `[cod]`.
- O comando e os valores viajam **separados** até o servidor: o valor nunca vira parte do SQL.

EXEMPLO EXPLICADO · UM PARÂMETRO

Os funcionários de um setor

consultas_rh.py

```python
from classBanco import Banco

banco = Banco()

cod_setor = int(input("Ver os funcionários de qual setor? "))
linhas = banco.consultar(
    """SELECT nome_funcionario, cargo_funcionario, salario_funcionario
       FROM funcionarios
       WHERE cod_setor = %s
       ORDER BY nome_funcionario""",
    (cod_setor,),
)
if len(linhas) == 0:
    print("Nenhum funcionário nesse setor.")
for nome, cargo, salario in linhas:
    print(f"{nome} | {cargo} | R$ {salario:,.2f}")
```

TERMINAL

```
> python consultas_rh.py
Ver os funcionários de qual setor? 3
Carla Mendes | Desenvolvedora | R$ 6,800.00
Diego Rocha | Suporte técnico | R$ 3,500.00
```

TERMINAL

```
> python consultas_rh.py
Ver os funcionários de qual setor? 9
Nenhum funcionário nesse setor.
```

- Aspas triplas no SQL: dá para quebrar a consulta em linhas, como no pgAdmin.
- `len(linhas) == 0`: o banco respondeu, mas sem nenhuma linha. Não é erro.
- O `:,.2f` formata o `Decimal` com duas casas, como o salário na UC 1.

EXEMPLO EXPLICADO • ILIKE E DOIS PARÂMETROS

Busca por nome e reajuste de salário

BUSCA POR PARTE DO NOME

```
termo = input("Buscar pelo nome: ")
linhas = banco.consultar(
    """SELECT nome_funcionario, cargo_funcionario
       FROM funcionarios
       WHERE nome_funcionario ILIKE %s""",
    ("%" + termo + "%",),
)
for nome, cargo in linhas:
    print(nome, "-", cargo)
```

TERMINAL

```
Buscar pelo nome: an
Ana Souza - Analista de RH
```

O `%` do ILIKE (Capítulo 3) vai **no valor**, montado no Python: `'%an%'`. No SQL fica só o `%s`.

DOIS PARÂMETROS E O ROWCOUNT

```
cod = int(input("Código do funcionário para reajustar: "))
novo_salario = float(input("Novo salário: R$ "))
quantidade = banco.executar(
    "UPDATE funcionarios SET salario_funcionario = %s WHERE cod_funcionario = %s",
    (novo_salario, cod),
)
if quantidade == 0:
    print("Nenhum funcionário com esse código.")
else:
    print("Salário atualizado.")
```

TERMINAL

```
Código do funcionário para reajustar: 4
Novo salário: R$ 3650
Salário atualizado.
```

TERMINAL

```
Código do funcionário para reajustar: 99
Novo salário: R$ 100
Nenhum funcionário com esse código.
```

SQL injection: o usuário reescreve o seu SQL

COM F-STRING: O VALOR COLADO NO TEXTO

```
nome = input("Nome: ")
banco.consultar(f"SELECT nome_funcionario, salario_funcionario
  FROM funcionarios WHERE nome_funcionario = '{nome}'")
```

DIGITADO • O WHERE QUE CHEGA AO BANCO	RESPOSTA
Ana Souza `WHERE nome_funcionario = 'Ana Souza'`	1 linha
' OR '1'='1 `WHERE nome_funcionario = '' OR '1'='1'`	**as 5 linhas, com todos os salários**
Joana D'Ávila `WHERE nome_funcionario = 'Joana D'Ávila'`	`syntax error at or near "Ávila"`

COM %S: O VALOR VAI SEPARADO

```
nome = input("Nome: ")
banco.consultar("""SELECT nome_funcionario, salario_funcionario
  FROM funcionarios WHERE nome_funcionario = %s""", (nome,))
```

DIGITADO	RESPOSTA
Ana Souza	1 linha
' OR '1'='1	`[]`: procurou alguém com esse nome esquisito
Joana D'Ávila	`[]`, sem erro

⚠ No DELETE, o mesmo truque apaga a tabela inteira. No teste, `f"DELETE FROM funcionarios WHERE cpf_funcionario = '{cpf}'"` com `' OR '1'='1` apagou os 5 funcionários. Regra do capítulo: **valor digitado entra só por %s**.

02 • PARA SABER

Duas formas que você vai encontrar por aí

LINHAS COMO DICIONÁRIO: DICT_ROW

```python
from psycopg.rows import dict_row

conexao = banco.conectar()
cursor = conexao.cursor(row_factory=dict_row)
cursor.execute("""SELECT nome_funcionario, salario_funcionario
                  FROM funcionarios WHERE cod_funcionario = 3""")
print(cursor.fetchall())
conexao.close()
```

```
TERMINAL
[{'nome_funcionario': 'Carla Mendes', 'salario_funcionario':
Decimal('6800.00')}]
```

Igual à lista de dicionários da UC 1. Neste capítulo seguimos com tuplas: o `for` com desempacotamento resolve com menos código.

A CONEXÃO COM WITH

```python
with banco.conectar() as conexao:
    conexao.execute(
        "DELETE FROM funcionarios WHERE cod_funcionario = %s", (6,))
# saiu do with: commit feito e conexão fechada
```

- No fim do bloco, o `with` faz o `commit` e fecha. Se deu erro, faz o `rollback` e fecha.
- É o mesmo papel do `try/finally`, escrito de outro jeito, como o `with open(...)` do `dados.json`.
- Neste capítulo usamos o `try/except/finally`: cada passo fica à vista.

PRÁTICA 2 • 10 MIN • INDIVIDUAL

Encontre o erro

Banco **cinema**, com `banco = Banco()`. Para cada trecho: dá erro, dá resultado errado ou está certo? Explique e corrija. **Um deles está certo.**

a

```
cod = 2
banco.consultar(
    "SELECT titulo_filme FROM filmes WHERE cod_filme = %s", (cod))
```

b

```
classif = "L"
banco.consultar("""SELECT titulo_filme FROM filmes
    WHERE classificacao_filme = '%s'""", (classif,))
```

c

```
banco.consultar(
    "SELECT titulo_filme FROM filmes WHERE cod_filme = %d", (cod,))
```

d

```
banco.executar(
    "UPDATE sessoes SET sala_sessao = %s WHERE cod_sessao = %s", (5,))
```

e

```
titulo = "Noite sem Fim"
banco.consultar(f"""SELECT duracao_filme FROM filmes
    WHERE titulo_filme = {titulo}""")
```

f

```
termo = "farol"
banco.consultar("""SELECT titulo_filme FROM filmes
    WHERE titulo_filme ILIKE %s""", (termo,))
```

O filme se chama *O Último Farol*.

g

```
conexao = banco.conectar()
cursor = conexao.cursor()
cursor.execute("DELETE FROM sessoes WHERE cod_sessao = %s", (6,))
conexao.close()
print("Sessão excluída!")
```

h

```
banco.consultar(
    "SELECT titulo_filme FROM filmes WHERE cod_filme = %s", [cod])
```

PRÁTICA 3 • 20 MIN • INDIVIDUAL

Consultas do Cinema Estrela

Copie o `classBanco.py` para a pasta **cinema**. No `consultas_cinema.py`, uma consulta depois da outra, todas com `%s`:

1. Pergunte a **classificação** e liste os filmes dela: título e duração.
2. Pergunte **parte do título** e mostre as sessões do filme: título, data (dd/mm/aaaa), horário (hh:mm) e sala. *JOIN + ILIKE.*
3. Pergunte o código de uma **sessão** e a nova sala, e mude. Use o `rowcount` e trate o `CheckViolation` com uma mensagem.
4. Pergunte o código de um **filme** e exclua. Trate o `ForeignKeyViolation` e o código que não existe.

TESTE	ESPERADO
1 • **L**	A Ilha dos Gatos (90 min) • Corrida nas Dunas (98 min)
2 • **dun**	Corrida nas Dunas | 07/11/2026 às 15:00 | sala 2 Corrida nas Dunas | 08/11/2026 às 16:30 | sala 4
3 • sessão **3**, sala **9**	mensagem de sala inválida, e nada muda
3 • sessão **99**, sala **2**	“Nenhuma sessão com esse código.”
4 • filme **1**	mensagem: o filme tem sessões
4 • filme **5**	“Filme excluído.”

Rode o `setup_cinema.py` antes de testar de novo: ele devolve o filme 5.

PENSE E RESPONDA O filme 5 saiu e o filme 1 não. Que regra do banco decidiu isso, e em qual tabela ela está?

SEÇÃO 03

CRUD com banco

03

O Sistema de Funcionários da UC 1, agora com os dados no PostgreSQL.

Da lista para a tabela

NA UC 1 • LISTA E DADOS.JSON	COM BANCO • TABELA FUNCIONARIOS
`funcionarios = carregar_dados()`	`banco = Banco()` : os dados ficam no servidor
`funcionarios.append(novo_funcionario)`	`INSERT INTO funcionarios (...) VALUES (%s, %s, ...)`
`for funcionario in funcionarios: print(...)`	`SELECT ... JOIN setores ... ORDER BY` e o `for` nas linhas
`funcionario["cargo"] = novo_cargo`	`UPDATE funcionarios SET ... WHERE cod_funcionario = %s`
`funcionarios.pop(num_funcionario - 1)`	`DELETE FROM funcionarios WHERE cod_funcionario = %s`
escolher pela **posição** na lista	escolher pelo **código**, a PK
`cpf_exist(cpf)` percorre a lista	o `UNIQUE` recusa, e o programa traduz o erro
`salvar_dados()` só ao sair	cada operação já grava: o `commit` está no `executar`

O menu, o `while True` e as funções ferramentas continuam iguais. Muda só **onde o dado mora**.

O main.py por partes

main.py

```python
import psycopg
from classBanco import Banco

banco = Banco()

# ---- Funções ferramentas ----
def ler_texto(mensagem): ...
def ler_inteiro(mensagem): ...
def ler_decimal(mensagem): ...
def gravar(sql, parametros): ...       # INSERT/UPDATE + mensagens

# ---- Operações CRUD ----
def cadastrar_funcionario(): ...
def ver_funcionarios(): ...
def escolher_funcionario(): ...
def alterar_funcionario(): ...
def remover_funcionario(): ...

# ---- Funcionalidade extra ----
def buscar_por_nome(): ...

# ---- Menu principal ----
while True:
    ...
```

A mesma organização sugerida na atividade da UC 1: ferramentas, CRUD, extra e menu.

- `import psycopg` continua: o `main.py` precisa dos tipos de erro (`psycopg.errors...`).
- Um objeto `banco` só, criado no início e usado por todas as funções.
- Nenhuma função abre conexão: quem conversa com a Aiven é a classe.

```
rh/
├── .env              DB_NAME=rh
├── classBanco.py
├── setup_rh.py       rode antes, uma vez
└── main.py           o sistema
```

O que o Python valida antes de ir ao banco

```python
def ler_texto(mensagem):
    while True:
        texto = input(mensagem).strip()
        if texto != "":
            return texto
        print("Preencha este campo.")


def ler_inteiro(mensagem):
    while True:
        try:
            return int(input(mensagem))
        except ValueError:
            print("Digite apenas números inteiros.")


def ler_decimal(mensagem):
    while True:
        try:
            return float(input(mensagem).replace(",", "."))
        except ValueError:
            print("Digite apenas números.")
```

- A mesma ideia do `while True` da UC 1: só sai quando o valor serve.
- `ler_decimal` aceita vírgula: `1500,00` vira `1500.0`.
- `.strip()` tira os espaços das pontas: só espaço conta como vazio.

Por que o Python cuida do vazio?

Para o banco, texto vazio `''` **não** é `NULL`. No teste, um funcionário com nome `''` passou pelo `NOT NULL` sem erro.

TERMINAL

```
Nome:
Preencha este campo.
Nome: Fábio Nunes
```

Ver funcionários: SELECT com JOIN

```python
def ver_funcionarios():
    print("LISTA DE FUNCIONÁRIOS")
    linhas = banco.consultar("""
        SELECT f.cod_funcionario, f.nome_funcionario, f.cargo_funcionario,
               f.salario_funcionario, s.nome_setor
        FROM funcionarios f
        JOIN setores s ON s.cod_setor = f.cod_setor
        ORDER BY f.nome_funcionario
    """)
    if len(linhas) == 0:
        print("Nenhum funcionário cadastrado.")
    for cod, nome, cargo, salario, setor in linhas:
        print(f"{cod}. {nome} - {cargo} - R$ {salario:,.2f} - {setor}")
```

TERMINAL

```
LISTA DE FUNCIONÁRIOS
1. Ana Souza - Analista de RH - R$ 4,200.00 - Administrativo
2. Bruno Lima - Vendedor - R$ 3,100.00 - Comercial
3. Carla Mendes - Desenvolvedora - R$ 6,800.00 - Tecnologia
4. Diego Rocha - Suporte técnico - R$ 3,500.00 - Tecnologia
5. Elisa Martins - Gerente comercial - R$ 7,500.00 - Comercial
```

- O JOIN do Capítulo 4 traz o **nome do setor** no lugar do código.
- Sem parâmetro: a lista não depende de nada digitado.
- No lugar do `contador` da UC 1, aparece o **código**: é ele que o usuário digita para alterar e remover.

Cadastrar: INSERT com parâmetros

```python
def cadastrar_funcionario():
    print("CADASTRO DE FUNCIONÁRIOS")
    nome = ler_texto("Nome: ")
    cpf = ler_texto("CPF (só números): ")
    cargo = ler_texto("Cargo: ")
    salario = ler_decimal("Salário: R$ ")
    print("Setores:")
    ver_setores()
    cod_setor = ler_inteiro("Código do setor: ")

    ok = gravar(
        """INSERT INTO funcionarios
             (nome_funcionario, cpf_funcionario, cargo_funcionario,
              salario_funcionario, cod_setor)
           VALUES (%s, %s, %s, %s, %s)""",
        (nome, cpf, cargo, salario, cod_setor),
    )
    if ok:
        print(f"{nome} foi cadastrado(a)!")
```

TERMINAL

```
CADASTRO DE FUNCIONÁRIOS
Nome: Fábio Nunes
CPF (só números): 67890123456
Cargo: Estagiário
Salário: R$ 1500,00
Setores:
  1. Administrativo
  2. Comercial
  3. Tecnologia
Código do setor: 3
Fábio Nunes foi cadastrado(a)!
```

- Cinco `%s`, cinco valores, na mesma ordem das colunas.
- O código não entra: o `GENERATED` deu o **6** ao Fábio.
- `ver_setores()` mostra os códigos válidos antes da pergunta.

EXEMPLO EXPLICADO • ERROS DO BANCO VIRAM MENSAGENS

gravar(): da regra do banco à mensagem

```python
def gravar(sql, parametros):
    # Roda o INSERT ou o UPDATE e traduz as regras do banco em mensagens
    try:
        banco.executar(sql, parametros)
        return True
    except psycopg.errors.UniqueViolation:
        print("CPF já cadastrado. Tente outro número.")
    except psycopg.errors.ForeignKeyViolation:
        print("Esse setor não existe.")
    except psycopg.errors.CheckViolation as erro:
        if erro.diag.constraint_name == "ck_cpf_tamanho":
            print("O CPF precisa ter 11 dígitos.")
        else:
            print("O salário precisa ser maior que zero.")
    return False
```

REGRA NO BANCO	ERRO NO PYTHON	MENSAGEM
`UNIQUE` do CPF	`UniqueViolation`	CPF já cadastrado.
`fk_func_setor`	`ForeignKeyViolation`	Esse setor não existe.
`ck_cpf_tamanho`	`CheckViolation`	O CPF precisa ter 11 dígitos.
`ck_salario_positivo`	`CheckViolation`	O salário precisa ser maior que zero.

- Dois CHECKs, um tipo de erro só: o `erro.diag.constraint_name` diz **qual regra** recusou. É o nome do `CONSTRAINT` do CREATE.
- A mesma função serve para o cadastrar e o alterar: as regras são as mesmas.
- `True` ou `False`: quem chamou sabe se gravou.

EXEMPLO EXPLICADO • QUATRO CADASTROS RECUSADOS

O banco disse não, o programa explicou

CPF DA ANA, REPETIDO

```
UniqueViolation
Nome: Gil Paz · CPF: 12345678901 · ... setor 2
CPF já cadastrado. Tente outro número.
```

CPF COM 10 DÍGITOS

```
CheckViolation · ck_cpf_tamanho
Nome: Gil Paz · CPF: 1234567890 · ... setor 2
O CPF precisa ter 11 dígitos.
```

SALÁRIO ZERO

```
CheckViolation · ck_salario_positivo
Nome: Gil Paz · ... Salário: R$ 0 · setor 2
O salário precisa ser maior que zero.
```

SETOR 9, QUE NÃO EXISTE

```
ForeignKeyViolation
Nome: Gil Paz · ... Salário: R$ abc
Digite apenas números.
Salário: R$ 3000 · setor 9
Esse setor não existe.
```

Repare no último: o `abc` foi barrado pelo Python (`ler_decimal`), antes de chegar ao banco. O setor 9, só o banco podia saber.

PENSE E RESPONDA O Fábio entrou com o código 6. Depois desses quatro cadastros recusados, o Gil é cadastrado direito. Que código ele recebe? (Lembre do GENERATED no Capítulo 4.)

Alterar: deixe vazio para manter

```python
def alterar_funcionario():
    print("ALTERAR FUNCIONÁRIO")
    funcionario = escolher_funcionario()
    if funcionario is None:
        return
    cod, nome, cargo, salario, cod_setor = funcionario
    print("Deixe vazio para manter o valor atual.")
    novo_nome = input(f"Nome [{nome}]: ").strip()
    novo_cargo = input(f"Cargo [{cargo}]: ").strip()
    novo_salario = input(f"Salário [{salario}]: ").strip()
    novo_setor = input(f"Setor [{cod_setor}]: ").strip()

    if novo_nome != "":
        nome = novo_nome
    if novo_cargo != "":
        cargo = novo_cargo
    try:
        if novo_salario != "":
            salario = float(novo_salario.replace(",", "."))
        if novo_setor != "":
            cod_setor = int(novo_setor)
    except ValueError:
        print("Salário e setor precisam ser números. Nada foi alterado.")
        return
    ok = gravar(
        """UPDATE funcionarios SET nome_funcionario = %s, cargo_funcionario = %s,
               salario_funcionario = %s, cod_setor = %s WHERE cod_funcionario = %s""",
        (nome, cargo, salario, cod_setor, cod),
    )
    if ok:
        print("Dados atualizados!")
```

TERMINAL

```
ALTERAR FUNCIONÁRIO
(a lista de funcionários)
Código do funcionário: 6
Deixe vazio para manter o valor atual.
Nome [Fábio Nunes]:
Cargo [Estagiário]: Desenvolvedor júnior
Salário [1500.00]: 2800
Setor [3]:
Dados atualizados!
```

- A linha do banco é desempacotada em variáveis com o valor **atual**.
- O que veio digitado substitui; o que veio vazio fica como estava.
- O UPDATE regrava tudo, com o código no último `%s`, o do WHERE.
- Setor 9? O mesmo `gravar` mostra “Esse setor não existe.”

Escolher e remover pelo código

```python
def escolher_funcionario():
    # Devolve a linha do funcionário escolhido, ou None se o código não existe
    ver_funcionarios()
    cod = ler_inteiro("Código do funcionário: ")
    linhas = banco.consultar(
        """SELECT cod_funcionario, nome_funcionario, cargo_funcionario,
                  salario_funcionario, cod_setor
           FROM funcionarios
           WHERE cod_funcionario = %s""",
        (cod,),
    )
    if len(linhas) == 0:
        print("Nenhum funcionário com esse código.")
        return None
    return linhas[0]


def remover_funcionario():
    print("REMOVER FUNCIONÁRIO")
    funcionario = escolher_funcionario()
    if funcionario is None:
        return
    cod = funcionario[0]
    nome = funcionario[1]

    resposta = input(f"Remover {nome}? (s/n) ").strip().lower()
    if resposta == "s":
        banco.executar("DELETE FROM funcionarios WHERE cod_funcionario = %s", (cod,))
        print(f"{nome} foi removido(a).")
    else:
        print("Nada foi removido.")
```

TERMINAL

```
REMOVER FUNCIONÁRIO
(a lista de funcionários)
Código do funcionário: 6
Remover Fábio Nunes? (s/n) s
Fábio Nunes foi removido(a).
```

TERMINAL

```
ALTERAR FUNCIONÁRIO
(a lista de funcionários)
Código do funcionário: 99
Nenhum funcionário com esse código.
```

- `escolher_funcionario` serve ao alterar e ao remover: busca pelo código e devolve a linha, ou `None`.
- A confirmação **s/n** é do Python: o DELETE não tem volta.

O menu, e quando a internet cai

```python
while True:
    print("""
SISTEMA RH · BANCO DE DADOS

1. Cadastrar funcionário
2. Ver funcionários
3. Alterar funcionário
4. Remover funcionário
5. Buscar por nome

0. Sair
""")
    op = input("Digite a opção desejada: ")

    try:
        if op == "1":
            cadastrar_funcionario()
        elif op == "2":
            ver_funcionarios()
        # ... 3, 4 e 5
        elif op == "0":
            print("Saindo do programa...")
            break
        else:
            print("Opção inválida.")
    except psycopg.OperationalError:
        print("Sem conexão com o banco. Confira a internet e o arquivo .env.")

    input("Tecle ENTER para continuar")
```

O menu é o mesmo da UC 1. A novidade é o `try` em volta das opções.

- Os erros de **regra** já são tratados dentro de cada função (`gravar`).
- Sobra um: o banco fora do alcance. Wi-Fi caiu, Aiven desligou o serviço gratuito parado. Qualquer opção pode esbarrar nele.
- Com o `except` no menu, o programa avisa e **continua**: a próxima opção tenta de novo.
- Não precisa mais do `salvar_dados()` ao sair: tudo já está no banco.

TERMINAL

```
Digite a opção desejada: 2
Sem conexão com o banco. Confira a internet e o arquivo .env.
Tecle ENTER para continuar
```

Quem valida: o Python ou o banco?

O PYTHON • A PRIMEIRA PORTA

- Campo vazio: `''` passa no NOT NULL.
- Texto onde devia ser número: `int("abc")`.
- Confirmação antes de apagar.
- Mensagens na hora, sem ir ao servidor.

O BANCO • A ÚLTIMA PORTA

- CPF repetido: só o banco conhece **todos** os CPFs.
- Setor que não existe: a FK.
- CPF com 11 dígitos, salário maior que zero: os CHECKs.
- Vale para **qualquer** programa: o seu, o pgAdmin, o app do colega.

Na UC 1, o `cpf_exist` percorria a lista. Com dois usuários cadastrando ao mesmo tempo, cada um olharia a própria lista. Com banco, a regra é **uma só**, no servidor: mesmo que o Python esqueça uma validação, a regra segura.

PRÁTICA 4 • 10 MIN • INDIVIDUAL

Qual mensagem aparece?

Banco **cinema** recém-montado pelo setup (5 filmes). Antes de rodar, escreva o que cada chamada mostra na tela. Depois rode e confira.

```python
def cadastrar_filme(titulo, duracao, classificacao):
    try:
        banco.executar(
            """INSERT INTO filmes (titulo_filme, duracao_filme, classificacao_filme)
               VALUES (%s, %s, %s)""",
            (titulo, duracao, classificacao),
        )
        print("Filme cadastrado:", titulo)
    except psycopg.errors.CheckViolation as erro:
        if erro.diag.constraint_name == "ck_duracao":
            print("Duração inválida.")
        else:
            print("Classificação inválida.")
    except psycopg.errors.NotNullViolation:
        print("Faltou um dado obrigatório.")
    except psycopg.Error:
        print("Erro no banco.")
    finally:
        print("---")
```

```python
cadastrar_filme("Maré Alta", 101, "12")      # a
cadastrar_filme("Maré Alta", 0, "12")        # b
cadastrar_filme("Maré Alta", 101, "13")      # c
cadastrar_filme(None, 101, "L")              # d
cadastrar_filme("Maré Alta", 101, "Livre")   # e
cadastrar_filme("Maré Alta", 101, "12")      # f
```

- Para cada letra: a mensagem **e** o que o `finally` mostra.
- No **e**, lembre do tipo de `classificacao_filme`.
- O programa está no repositório: `cinema/cadastrar_filme.py`.

PENSE E RESPONDA A chamada f cadastrou outra “Maré Alta”, igual à da letra a. Que regra faltou no banco para recusar o filme repetido?

SEÇÃO 04

Prática guiada: Loja Conecta

04

O banco do JOIN do Capítulo 4 ganha um sistema: produtos, pedidos com RETURNING e a transação que grava tudo ou nada.

PRÁTICA GUIADA • INDIVIDUAL • 6 PASSOS

O sistema da Loja Conecta

O mesmo banco do JOIN, com os mesmos dados. Agora ele ganha um programa: cada passo acrescenta uma parte e termina com um teste de **“deu certo se…”**.



PASSO	O QUE ENTRA	TEMPO
1	`setup_loja.py` : o banco com códigos gerados	15 min
2	`classBanco.py` com o método `inserir` (RETURNING)	10 min
3	Ver e cadastrar produtos	20 min
4	Alterar e remover produtos	15 min
5	Novo pedido: RETURNING e transação	30 min
6	Ver um pedido e os pedidos de um cliente	15 min

O MENU NO FIM DO PASSO 6

```
LOJA CONECTA · SISTEMA

1. Ver produtos
2. Cadastrar produto
3. Alterar preço e estoque
4. Remover produto
5. Novo pedido
6. Ver pedido
7. Pedidos de um cliente

0. Sair
```

PRÁTICA GUIADA • PASSO 1 • 15 MIN

setup_loja.py: agora o banco gera os códigos

```
cursor.execute("""
    CREATE TABLE produtos (
      cod_produto  integer  GENERATED ALWAYS AS IDENTITY (START WITH 10) PRIMARY KEY,
      nome_produto varchar(80) NOT NULL,
      ...
    )
""")
cursor.execute("""
    INSERT INTO produtos (nome_produto, preco, estoque, cod_categoria)
    VALUES ('Notebook Lume 14',  3499.00,  5, 1),
           ('Monitor 24 pol',     899.90,  8, 1),
           ...
""")
```

No Capítulo 4, os códigos vinham prontos nos dados. Agora o sistema vai **cadastrar** produtos e pedidos: ninguém deve escolher o próximo número. O `START WITH` faz o contador começar onde o Capítulo 4 começava: os dados ficam iguais.

1. Pasta **loja** com `.env` (`DB_NAME=loja`), `.env.exemplo` e `.gitignore`.
2. No molde do `setup_rh.py`, monte o setup com o SQL de `Capitulo 4 - Banco Loja Conecta.sql`.
3. Em toda PK: `GENERATED ALWAYS AS IDENTITY`. Produtos com `(START WITH 10)`, pedidos com `(START WITH 100)`.
4. Tire os códigos dos INSERT. A **ordem** das linhas decide o código de cada uma.
5. Rode **duas vezes**.

DEU CERTO SE…

No pgAdmin: produtos de **10 a 16**, pedidos de **100 a 107**, 11 itens. Os JOINs do Capítulo 4 dão o mesmo resultado.

PRÁTICA GUIADA • PASSO 2 • 10 MIN

classBanco.py: o método inserir

Copie o `classBanco.py` do RH para a pasta **loja** e acrescente, dentro da classe:

```
    # INSERT ... RETURNING: grava e devolve o código que o banco gerou
    def inserir(self, sql, parametros=None):
        conexao = self.conectar()
        try:
            cursor = conexao.cursor()
            cursor.execute(sql, parametros)
            codigo = cursor.fetchone()[0]
            conexao.commit()
            return codigo
        finally:
            conexao.close()
```

- O `RETURNING` (Capítulos 3 e 4) faz o INSERT **responder** com a linha que entrou, como um SELECT.
- `fetchone()` pega essa linha; o `[0]`, a primeira coluna: o código.

TESTE RÁPIDO • TESTE_INSERIR.PY

```
from classBanco import Banco

banco = Banco()
cod = banco.inserir(
    """INSERT INTO categorias (nome_categoria)
       VALUES (%s) RETURNING cod_categoria""",
    ("Casa",),
)
print("Nova categoria:", cod)
```

TERMINAL

```
> python teste_inserir.py
Nova categoria: 5
```

DEU CERTO SE…

Apareceu o **5**: as categorias 1 a 4 vieram do setup. Rode o `setup_loja.py` de novo para voltar aos dados originais.

PRÁTICA GUIADA • PASSO 3 • 20 MIN • 1 DE 2

main.py: ver produtos

```python
def ver_produtos():
    print("PRODUTOS")
    linhas = banco.consultar("""
        SELECT p.cod_produto, p.nome_produto, p.preco, p.estoque, c.nome_categoria
        FROM produtos p
        LEFT JOIN categorias c ON c.cod_categoria = p.cod_categoria
        ORDER BY p.cod_produto
    """)
    for cod, nome, preco, estoque, categoria in linhas:
        if categoria is None:
            categoria = "sem categoria"
        print(f"{cod}. {nome} - R$ {preco:,.2f} - estoque {estoque} - {categoria}")


def buscar_produto(cod_produto):
    # Devolve (cod, nome, preco, estoque) ou None se o produto não existe
    linhas = banco.consultar(
        "SELECT cod_produto, nome_produto, preco, estoque FROM produtos WHERE cod_produto = %s",
        (cod_produto,),
    )
    if len(linhas) == 0:
        return None
    return linhas[0]
```

1. Comece o `main.py` com os imports, `banco = Banco()` e as funções ferramentas do RH.
2. Escreva `ver_produtos` e `buscar_produto`.
3. Monte o menu com a opção 1 e o `0. Sair`.

Por que **LEFT** JOIN? O Cabo HDMI não tem categoria: no INNER, ele sumiria da lista (Capítulo 4). A categoria dele chega como `None`.

DEU CERTO SE…

OPÇÃO 1

```
10. Notebook Lume 14 - R$ 3,499.00 - estoque 5 - Informática
...
16. Cabo HDMI - R$ 39.90 - estoque 50 - sem categoria
```

main.py: cadastrar produto com RETURNING

```python
def ler_inteiro_ou_vazio(mensagem):
    # Devolve um número, ou None se a pessoa deixar vazio (None vira NULL no banco)
    while True:
        texto = input(mensagem).strip()
        if texto == "":
            return None
        try:
            return int(texto)
        except ValueError:
            print("Digite um número inteiro ou deixe vazio.")


def cadastrar_produto():
    print("CADASTRO DE PRODUTO")
    nome = ler_texto("Nome: ")
    preco = ler_decimal("Preço: R$ ")
    estoque = ler_inteiro("Estoque: ")
    print("Categorias:")
    ver_categorias()
    cod_categoria = ler_inteiro_ou_vazio("Código da categoria (vazio = sem categoria): ")

    try:
        cod = banco.inserir(
            """INSERT INTO produtos (nome_produto, preco, estoque, cod_categoria)
               VALUES (%s, %s, %s, %s)
               RETURNING cod_produto""",
            (nome, preco, estoque, cod_categoria),
        )
        print(f"Produto cadastrado com o código {cod}.")
    except psycopg.errors.ForeignKeyViolation:
        print("Essa categoria não existe.")
    except psycopg.errors.CheckViolation:
        print("Preço e estoque não podem ser negativos.")
```

`ver_categorias()` é como o `ver_setores()` do RH. Categoria vazia vira `None`, e o banco grava `NULL`: a FK aceita.

TESTE • OPÇÃO 2	ESPERADO
Teclado mecânico • 249,90 • 15 • categoria 3	Produto cadastrado com o código **17**.
Headset • 199 • 10 • categoria 9	Essa categoria não existe.
Headset • −5 • 10 • vazio	Preço e estoque não podem ser negativos.

DEU CERTO SE…

O teclado aparece na opção 1 como **17**, em Acessórios. Os dois Headsets não aparecem.

PRÁTICA GUIADA • PASSO 4 • 15 MIN

Alterar e remover produtos

```python
def remover_produto():
    print("REMOVER PRODUTO")
    ver_produtos()
    cod = ler_inteiro("Código do produto: ")
    try:
        quantidade = banco.executar(
            "DELETE FROM produtos WHERE cod_produto = %s", (cod,))
        if quantidade == 0:
            print("Produto não encontrado.")
        else:
            print("Produto removido.")
    except psycopg.errors.ForeignKeyViolation:
        print("Esse produto já foi vendido: ele fica no cadastro, por causa dos pedidos.")
```

ALTERAR_PRODUTO • OPÇÃO 3

No molde do `alterar_funcionario`: busque com `buscar_produto`, mostre preço e estoque atuais, "deixe vazio para manter", e faça o UPDATE com três `%s`. Trate o `CheckViolation`.

TESTE	ESPERADO
3 • produto 17 • preço 229,90 • estoque vazio	Teclado mecânico atualizado.
4 • produto 16, Cabo HDMI	Esse produto já foi vendido…
4 • produto 15, Mouse sem fio	Produto removido.
4 • produto 99	Produto não encontrado.

PENSE E RESPONDA O Mouse sem fio (15) saiu e o Cabo HDMI (16) não. Qual FK do Capítulo 4 decidiu, e em qual pedido está o cabo?

PRÁTICA GUIADA • PASSO 5 • A IDEIA

Um pedido são várias gravações, e todas ou nenhuma

`INSERT INTO pedidos (cod_cliente) VALUES (%s) RETURNING cod_pedido`
o banco gera o **108**, e o RETURNING devolve

↓ para cada item

`INSERT INTO itens_pedido` (108, produto, qtd, preço)

`UPDATE produtos SET estoque = estoque - qtd`

↓

`commit()`: o pedido, os itens e a baixa no estoque, juntos

algum passo falhou? **`rollback()`**: nada fica gravado

- Sem o `RETURNING`, o programa não saberia o número do pedido para gravar os itens.
- Por que não usar o `banco.executar` em cada passo? Cada chamada abre, grava e fecha: seriam várias transações. Se o 3º item falhasse, o pedido ficaria **pela metade**, com o estoque dos dois primeiros já baixado.
- Aqui todas as gravações usam **uma conexão só**: `banco.conectar()`, com o `try/except/finally` da Seção 01.
- Quem impede vender mais do que tem é o `ck_estoque` do Capítulo 4: o UPDATE que deixaria o estoque negativo é recusado.

Isso é uma **transação**: um conjunto de comandos que o banco trata como um só. É o tudo ou nada da Seção 01, agora com uma razão de negócio.

novo_pedido: montar a lista de itens

```python
def ver_clientes():
    for cod, nome in banco.consultar("SELECT cod_cliente, nome_cliente FROM clientes ORDER BY cod_cliente"):
        print(f"  {cod}. {nome}")


def novo_pedido():
    print("NOVO PEDIDO")
    print("Clientes:")
    ver_clientes()
    cod_cliente = ler_inteiro("Código do cliente: ")

    # 1) Montar a lista de itens na memória, antes de gravar qualquer coisa
    itens = []
    while True:
        cod_produto = ler_inteiro("Código do produto (0 para fechar o pedido): ")
        if cod_produto == 0:
            break
        produto = buscar_produto(cod_produto)
        if produto is None:
            print("Produto não encontrado.")
            continue
        qtd = ler_inteiro(f"Quantidade de {produto[1]}: ")
        itens.append((cod_produto, qtd, produto[2]))

    if len(itens) == 0:
        print("Pedido sem itens: nada foi gravado.")
        return

    # 2) Gravar tudo numa transação só (próximos slides)
```

- Primeiro o programa só **pergunta**: nada vai ao banco enquanto a pessoa escolhe.
- Cada item é uma tupla: `(produto, quantidade, preço)`.
- O preço vem do cadastro (`produto[2]`) e vai para `preco_unit` : o preço do dia da venda, mesmo que o produto mude depois.
- O `0` fecha o pedido. Nenhum item? Nada a gravar.

PRÁTICA GUIADA • PASSO 5 • 2 DE 3

novo_pedido: gravar numa transação

```
    # 2) Gravar tudo numa transação só: pedido, itens e baixa no estoque
    conexao = banco.conectar()
    try:
        cursor = conexao.cursor()
        cursor.execute(
            "INSERT INTO pedidos (cod_cliente) VALUES (%s) RETURNING cod_pedido",
            (cod_cliente,),
        )
        cod_pedido = cursor.fetchone()[0]

        for cod_produto, qtd, preco in itens:
            cursor.execute(
                """INSERT INTO itens_pedido (cod_pedido, cod_produto, qtd, preco_unit)
                   VALUES (%s, %s, %s, %s)""",
                (cod_pedido, cod_produto, qtd, preco),
            )
            cursor.execute(
                "UPDATE produtos SET estoque = estoque - %s WHERE cod_produto = %s",
                (qtd, cod_produto),
            )

        conexao.commit()
        print(f"Pedido {cod_pedido} gravado com {len(itens)} item(ns).")

    # os except e o finally: próximo slide
```

- `banco.conectar()`, e não `banco.executar`: as gravações precisam da **mesma** conexão.
- O RETURNING responde com o código do pedido novo, e o `fetchone()[0]` pega esse código.
- Cada item usa o `cod_pedido` que acabou de voltar.
- Um `commit()` só, **depois** do `for`: até ali, nada é definitivo.

PRÁTICA GUIADA • PASSO 5 • 3 DE 3

novo_pedido: quando uma gravação falha

```
    except psycopg.errors.ForeignKeyViolation:
        conexao.rollback()
        print("Cliente não encontrado. Nada foi gravado.")
    except psycopg.errors.CheckViolation as erro:
        conexao.rollback()
        if erro.diag.constraint_name == "ck_estoque":
            print("Estoque insuficiente. Nada foi gravado.")
        else:
            print("A quantidade precisa ser maior que zero. Nada foi gravado.")
    except psycopg.errors.UniqueViolation:
        conexao.rollback()
        print("O mesmo produto entrou duas vezes: some as quantidades. Nada foi gravado.")
    finally:
        conexao.close()
```

REGRA (CAPÍTULO 4)	QUANDO
`fk_ped_cliente`	o cliente não existe
`ck_estoque`	a baixa deixaria o estoque negativo
`ck_qtd`	quantidade zero ou negativa
`uq_item_pedido_produto`	o mesmo produto duas vezes no pedido

- Cada `except` chama `rollback()`: o pedido, os itens já inseridos e as baixas no estoque são desfeitos.
- O `finally` fecha a conexão em qualquer caso.

PRÁTICA GUIADA • PASSO 5 • TESTES

Um pedido que passa e quatro que não passam

OPÇÃO 5 • NA ORDEM	ESPERADO	POR QUÊ
Cliente 6 (Gustavo) • 2 × Fone Bluetooth (13) • 1 × Teclado mecânico (17)	**Pedido 108 gravado com 2 item(ns).**	o RETURNING devolveu o 108
Cliente 2 • 10 × Notebook (10): o estoque é 5	Estoque insuficiente. Nada foi gravado.	`ck_estoque`
Cliente 99 • 1 × Fone (13)	Cliente não encontrado. Nada foi gravado.	`fk_ped_cliente`
Cliente 2 • 1 × Fone (13) • 2 × Fone (13)	O mesmo produto entrou duas vezes…	UNIQUE do par
Cliente 2 • 0 × Smartphone (12)	A quantidade precisa ser maior que zero…	`ck_qtd`
Cliente 2 • 1 × Capa para celular (14)	**Pedido 113 gravado com 1 item(ns).**	?

DEU CERTO SE…

Na opção 1: o Notebook continua com estoque **5**, o Fone foi para **23** e o Teclado para **14**. No pgAdmin, não existe nenhum pedido entre 109 e 112, nem item órfão.

PENSE E RESPONDA O último pedido saiu com o número 113, e não 109. Para onde foram os números 109 a 112, se nenhum desses pedidos foi gravado?

PRÁTICA GUIADA • PASSO 6 • 15 MIN

Ver um pedido, com o total

```python
def ver_pedido():
    print("VER PEDIDO")
    cod_pedido = ler_inteiro("Código do pedido: ")
    cabecalho = banco.consultar(
        """SELECT p.cod_pedido, c.nome_cliente, p.data_pedido, p.status
           FROM pedidos p
           JOIN clientes c ON c.cod_cliente = p.cod_cliente
           WHERE p.cod_pedido = %s""",
        (cod_pedido,),
    )
    if len(cabecalho) == 0:
        print("Pedido não encontrado.")
        return
    cod, cliente, data, status = cabecalho[0]
    print(f"Pedido {cod} · {cliente} · {data:%d/%m/%Y} · {status}")

    itens = banco.consultar(
        """SELECT pr.nome_produto, i.qtd, i.preco_unit, i.qtd * i.preco_unit AS subtotal
           FROM itens_pedido i
           JOIN produtos pr ON pr.cod_produto = i.cod_produto
           WHERE i.cod_pedido = %s
           ORDER BY i.cod_item""",
        (cod_pedido,),
    )
    total = 0
    for nome, qtd, preco_unit, subtotal in itens:
        print(f"  {qtd} x {nome} (R$ {preco_unit:,.2f}) = R$ {subtotal:,.2f}")
        total = total + subtotal
    if len(itens) == 0:
        print("  (sem itens)")
    print(f"Total: R$ {total:,.2f}")
```

OPÇÃO 6

```
Código do pedido: 108
Pedido 108 · Gustavo Ramos · 29/10/2026 · aberto
  2 x Fone Bluetooth (R$ 199.90) = R$ 399.80
  1 x Teclado mecânico (R$ 229.90) = R$ 229.90
Total: R$ 629.70
```

OPÇÃO 6

```
Código do pedido: 107
Pedido 107 · Mariana Costa · 05/10/2026 · aberto
  (sem itens)
Total: R$ 0.00
```

O **subtotal** sai do SQL (Capítulo 4); o **total**, do Python, somando no `for`. A data do 108 é a do dia em que o pedido foi feito (`DEFAULT CURRENT_DATE`).

PEDIDOS_DO_CLIENTE · OPÇÃO 7

Pergunte o cliente e liste código, data e status dos pedidos dele. Mariana (1): pedidos 100, 102 e 107.

PRÁTICA GUIADA • PARA IR ALÉM

Desafios da Loja Conecta

1 Cancelar pedido

Mude o status para `cancelado` e **devolva** ao estoque a quantidade de cada item. Tudo numa transação: se uma parte falhar, nada muda.

2 Cadastrar cliente

Com `inserir` e RETURNING: mostre o código novo. E-mail repetido? Mensagem amigável, pelo `UniqueViolation`.

3 Buscar produto pelo nome

Parte do nome, com `ILIKE %s` e o `%` no valor. Mostre preço e estoque.

4 Avançar o status

Pergunte o pedido e o novo status. Valor fora da lista? Trate o `ck_status` do Capítulo 4.

O gabarito dos 6 passos está no repositório: `Capitulo 5 - Python e PostgreSQL/loja`. Compare depois de fazer, não antes.

ATIVIDADE FINAL • INDIVIDUAL • TEMA SORTEADO

Do briefing ao sistema

Cada um sorteia um tema e constrói o próprio banco e o próprio sistema: da modelagem ao CRUD em Python.

1 Modelar
DER e MER no draw.io, a partir do briefing

→

2 Banco
o setup.py na Aiven: tabelas, regras e dados

→

3 CRUD
classBanco.py e o menu de um cadastro

→

4 Movimento
o registro com itens, RETURNING e transação

Usa os Capítulos 1 a 5: entidades, cardinalidade, DER, MER, formas normais, restrições, scripts, JOIN, .env, psycopg, parâmetros e transação.

ATIVIDADE FINAL • O QUE ENTREGAR

Um sistema completo, no seu tema

1 • MODELAGEM

- Entidades, relacionamentos e cardinalidade do briefing.
- DER em Chen, sem atributos.
- MER com todas as tabelas, PK e FK, ligadas com 1 e N.
- O N:N do movimento vira tabela de itens.

2 • BANCO • SETUP.PY

- DROP, CREATE e INSERT com `try/except/finally`. Roda duas vezes seguidas.
- Toda PK com `GENERATED ALWAYS`.
- Toda FK, CHECK e UNIQUE com nome: uma regra para cada exigência do briefing.
- Dados para testar: 5 linhas em cada cadastro e 3 movimentos com itens.

3 E 4 • SISTEMA • MAIN.PY

- `classBanco.py` com `consultar`, `executar` e `inserir`.
- CRUD completo de um cadastro que tem FK (ex.: sabores → categorias).
- Cada regra do banco vira uma mensagem.
- O movimento: cabeçalho com RETURNING e itens, numa transação só.
- Ver um movimento com os itens e o total, e uma consulta com JOIN e parâmetro.

Entrega: repositório no GitHub com o DER/MER (imagem ou `.drawio`), o `.env.exemplo`, o `.gitignore` e os `.py`. O `.env` **não** vai: confira no `git status`.

ATIVIDADE FINAL • TEMAS PARA O SORTEIO • 1 A 6

Os temas

1 Pizzaria Forno Bom

Os sabores (nome, preço) de cada categoria: tradicional, especial ou doce. Os clientes têm telefone único e endereço. Cada pedido é de um cliente e tem vários sabores, cada um com tamanho (P, M ou G) e quantidade.

Movimento: pedido com os sabores

2 Farmácia Bem-Estar

Os remédios (nome, preço, estoque, se exige receita) de cada laboratório. Os clientes têm CPF único. Cada venda é de um cliente e tem vários remédios com quantidade. O estoque nunca fica negativo.

Movimento: venda com os remédios

3 Pousada Mar Azul

Os quartos (número único, andar) de cada tipo, com o valor da diária. Os hóspedes têm CPF único. Cada reserva é de um hóspede, com entrada e saída (a saída depois da entrada), e pode incluir vários quartos.

Movimento: reserva com os quartos

4 Pet Feliz • Banho e Tosa

Os tutores (nome, telefone único) e os pets de cada um (nome, espécie: cão ou gato, porte P, M ou G). Os serviços têm preço. Cada agendamento é de um pet, com data e horário, e inclui vários serviços.

Movimento: agendamento com os serviços

5 Salão Beleza Pura

Os serviços (nome, duração em minutos, preço) de cada categoria: cabelo, unhas ou estética. Os clientes têm telefone único. Cada atendimento é de um cliente, numa data, e inclui vários serviços. Duração e preço maiores que zero.

Movimento: atendimento com os serviços

6 Cantina da Escola

Os produtos (nome, preço, estoque) de cada categoria: salgados, bebidas e doces. Os alunos têm matrícula única e turma. Cada pedido é de um aluno e tem vários produtos com quantidade. O estoque nunca fica negativo.

Movimento: pedido com os produtos

Os temas

7 Sebo Página Dois

Os livros (título, autor, preço, quantidade, estado: bom, regular ou ruim) de cada gênero. Os clientes têm e-mail único. Cada venda é de um cliente e tem vários livros com quantidade. O estoque nunca fica negativo.

Movimento: venda com os livros

8 Restaurante Sabor da Casa

Os pratos (nome, preço) de cada categoria: entrada, principal ou sobremesa. As mesas têm número único e de 1 a 12 lugares. Cada comanda é de uma mesa, numa data, está aberta ou fechada e tem vários pratos com quantidade.

Movimento: comanda com os pratos

9 Locadora Rota Livre

Os carros (placa única, modelo, ano) de cada categoria, com o valor da diária. Os clientes têm CNH única. Cada locação é de um cliente, com retirada e devolução (a devolução depois da retirada), e pode incluir vários carros.

Movimento: locação com os carros

10 Floricultura Jardim

Os arranjos (nome, preço, estoque) de cada tipo: buquê, cesta ou vaso. Os clientes têm telefone único. Cada encomenda é de um cliente, com endereço e data de entrega (nunca antes do dia da encomenda), e tem vários arranjos.

Movimento: encomenda com os arranjos

11 Lava-jato Brilho Total

Os clientes (nome, telefone único) e os veículos de cada um (placa única, modelo, tipo: carro, moto ou caminhonete). Os serviços têm preço. Cada atendimento é de um veículo, numa data, e inclui vários serviços.

Movimento: atendimento com os serviços

12 Agência Viaje Bem

Os pacotes (destino, data de saída, preço, vagas) de cada tipo: praia, serra ou cidade. Os clientes têm CPF único. Cada reserva é de um cliente e inclui um ou mais pacotes, com o número de pessoas. As vagas nunca ficam negativas.

Movimento: reserva com os pacotes

Os temas

13 Doceria Doce Encontro

Os doces e bolos (nome, preço, estoque) de cada categoria. Os clientes têm telefone único. Cada encomenda é de um cliente, com data de retirada, e tem vários itens com quantidade maior que zero. O estoque nunca fica negativo.

Movimento: encomenda com os doces

14 Água & Gás Express

Os produtos (galão de água, botijão de gás: preço e estoque). Os bairros atendidos têm taxa de entrega. Os clientes moram num bairro e têm telefone único. Cada pedido é de um cliente e tem vários produtos com quantidade.

Movimento: pedido com os produtos

15 Brechó Segunda Chance

As peças (descrição, tamanho de PP a GG, preço, estado: novo, seminovo ou usado) de cada categoria. Os clientes têm e-mail único. Cada venda é de um cliente e tem várias peças. Cada peça é única: só pode ser vendida uma vez.

Movimento: venda com as peças

16 Ótica Visão Clara

Os produtos (armação, lente ou óculos de sol: preço e estoque) de cada marca. Os clientes têm CPF único. Cada venda é de um cliente e tem vários produtos com quantidade. O estoque nunca fica negativo.

Movimento: venda com os produtos

17 Locadora de Jogos Joystick

Os jogos (título, preço do aluguel, cópias disponíveis) de cada plataforma. Os clientes têm e-mail único. Cada aluguel é de um cliente, com a devolução prevista, e inclui vários jogos. As cópias nunca ficam negativas.

Movimento: aluguel com os jogos

18 Hortifruti Feira Verde

Os produtos (nome, preço por kg, estoque em kg) de cada seção: frutas, verduras ou legumes. Os clientes têm telefone único. Cada venda é de um cliente e tem vários produtos, cada um com o peso em kg, maior que zero e com casas decimais.

Movimento: venda com os produtos

ATIVIDADE FINAL • ANTES DE ENTREGAR

Confira o seu sistema

MODELAGEM

- [ ] Toda entidade do briefing virou tabela.
- [ ] Toda FK fica no lado N.
- [ ] O movimento tem a tabela de itens, com as duas FKs.

BANCO

- [ ] `setup.py` roda duas vezes seguidas, sem erro.
- [ ] Toda PK com GENERATED ALWAYS.
- [ ] Cada exigência do briefing tem uma regra com nome.
- [ ] Testei um dado errado para cada regra.

SISTEMA

- [ ] O `.env` não aparece no `git status`.
- [ ] Todo valor digitado entra por `%s`.
- [ ] Movimento com erro não deixa nada pela metade.
- [ ] Nenhuma opção do menu termina em Traceback.

Sugestão: comece pela modelagem e mostre ao professor antes de escrever o `setup.py`. Um MER certo poupa horas de ALTER TABLE.

CAPÍTULO 5 • CHECKLIST

Confira antes de chamar o professor

CONEXÃO

- [ ] `.env` com os 6 dados da Aiven, na pasta do programa.
- [ ] `.gitignore` com `.env`, e o `.env.exemplo` sem senha.
- [ ] `testar_conexao.py` respondeu com o nome do banco.
- [ ] Setup roda duas vezes e fecha a conexão no `finally`.

CLASSE E PARÂMETROS

- [ ] `classBanco.py` na pasta de cada projeto.
- [ ] Todo valor digitado com `%s`, sem aspas.
- [ ] Um valor só? `(valor,)`, com a vírgula.
- [ ] Nenhuma f-string montando SQL.
- [ ] Práticas 1 a 3 no banco cinema.

CRUD E LOJA

- [ ] Cada regra do banco vira uma mensagem.
- [ ] Pedido gravado inteiro ou nada.
- [ ] `RETURNING` devolve o código novo.
- [ ] Os 6 passos da Loja com o “deu certo se…” conferido.

Resumo

.env
NOME=valor, um por linha
fica fora do GitHub
o .env.exemplo vai

Ler o .env
load_dotenv()
os.getenv("DB_HOST")
texto, ou None

Conectar
psycopg.connect(host,
port, dbname, user,
password, sslmode)

Cursor
execute(sql, parametros)
fetchone() · fetchall()
rowcount

Transação
commit() grava
rollback() desfaz
close() sem commit desfaz

try • except • finally
except do específico
para o geral
finally fecha a conexão

Tipos de erro
OperationalError
UniqueViolation · CheckViolation
ForeignKeyViolation

Classe Banco
consultar → linhas
executar → rowcount
inserir → código novo

Parâmetros
%s sem aspas
(valor,) com vírgula
nunca f-string no SQL

Tipos
numeric → Decimal
date → datetime.date
NULL ↔ None

RETURNING
INSERT ... RETURNING cod
cursor.fetchone()[0]
o código gerado

Pedido
1 conexão, vários execute
commit no fim
erro → rollback

Armadilhas: .env salvo como .env.txt • senha no código • esquecer o commit • (cod) sem vírgula • '%s' com aspas • f-string no SQL • except geral antes do específico • close() fora do finally • banco.executar em cada passo do pedido

BANCO DE DADOS · CAPÍTULO CONCLUÍDO

Seu programa fala com o banco!

Sorteie o seu tema e mãos à obra. Dúvidas? Discord aberto.

DISCORD
discord.gg/wt5CVZZWJs

DRIVE
drive.google.com/drive/folders/1hOl0DaPeAor7gnhKBUIlZ5n8lLRDNvUY

GITHUB
github.com/TarikPonciano/Programador-de-Sistema-SENAC

psycopg.org/psycopg3/docs — o guia do psycopg

pypi.org/project/python-dotenv — o .env

console.aiven.io — o seu serviço